MiniLang Compiler Visualizer

Report 1 — Design, Architecture and Compiler Theory

1. Project Overview

1.1 Problem Statement

Compiler design is one of those subjects that reads perfectly well on paper and still feels distant until you actually build one. A typical course walks through lexical analysis, parsing, semantic checking, intermediate code, optimisation and code generation as six more or less separate chapters, each with its own diagrams and worked examples. What rarely gets shown is how these six pieces actually fit together when they are run one after another on a real piece of source code — how a stream of characters becomes a token, how those tokens become a parse tree, how that tree becomes three-address code, and how that code eventually turns into something a machine can run. Most student compilers also share a second, more practical problem: the moment the parser meets one syntax mistake, it either stops dead or keeps going and reports several more mistakes that do not actually exist in the program. Neither behaviour is very forgiving to the person who wrote the code, and neither reflects how a real compiler such as GCC or javac is expected to behave.

1.2 Objectives

With that problem in mind, this project set out to do four things. First, build a complete compiler, phase by phase, for a small but real programming language of our own design, called MiniLang — not a toy example confined to one chapter, but one pipeline that takes a MiniLang program all the way from source text to an executable result. Second, make every intermediate structure the compiler builds along the way — the token stream, the parse tree, the symbol table, the three-address code, the control-flow graph, the optimised code and the final assembly — visible in a browser, instead of locked away as print statements or debugger state that only the person running the program ever sees. Third, use this pipeline as a vehicle to actually implement, rather than just describe, the classical algorithms a compiler-design course covers: deterministic scanning, predictive parsing driven by the structure of the grammar, scope-based symbol resolution, data-flow based optimisation, and stack-machine code generation with a proper activation record. Fourth, lay the groundwork for a syntax-error recovery scheme that reports a mistake precisely instead of cascading into a wall of unrelated errors; this report documents the pipeline that such a recovery scheme is meant to sit on top of.

1.3 Scope

The scope of the project is deliberately bounded so that depth was not traded away for breadth. In scope is a full six-phase compiler — lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimisation, and target code generation — for MiniLang, a small statically typed language with functions, four primitive types, the usual arithmetic and logical operators, conditionals, loops and recursion. Also in scope is a small stack-based virtual machine that actually executes the generated assembly and produces real output, and a browser front end that renders every phase's output live as the program is compiled. Out of scope, by design, is anything that belongs to an industrial compiler rather than an educational one: there is no register allocation, no instruction scheduling, no real target architecture such as x86 or ARM, no arrays, structs or generics, and no standard library beyond a single print statement. The project also does not attempt to compile an existing language such as C or Java — MiniLang is our own grammar, built specifically so that every phase of the course could be demonstrated cleanly without inheriting the enormous ambiguity and context-sensitivity of a real-world language's grammar.

1.4 Proposed Approach

The approach taken here rests on one central decision: build the actual compiler as a hand-written recursive-descent parser, and keep the theoretical, table-driven side of parsing — FIRST and FOLLOW sets, LL(1) tables, LR(0) item sets, SLR and LALR construction — as a separate, independent piece of the system rather than forcing the whole compiler through a parsing table. A table-driven parser is the right thing to study, but it is a poor foundation for good error messages, because a table only knows that no entry exists for the current state and symbol; it has no notion of what the programmer was probably trying to write. Recursive descent, on the other hand, lets every parsing function describe exactly what it expects next, which is what later makes precise, human-readable error messages possible. The second decision that shapes the whole system is the contract between the backend and the browser: a single compile request returns one JSON object holding the output of every phase at once — tokens, parse tree, symbol table, three-address code, control-flow graph, optimisation log and assembly — so the frontend's only job is to render what it is given. There is no client-side state machine deciding what to ask for next, which removes an entire category of bugs before they can occur. The sections that follow describe the architecture, the modules, the grammar and the algorithms that this approach is built from, phase by phase.

2. System Architecture

The system is split into two halves that talk to each other over plain HTTP: a browser front end that the user actually sees and types into, and a Python backend that does every bit of the actual compiling. Nothing in between needs a build step — the front end is one HTML page plus two hand-written JavaScript files, loaded by the browser exactly as written, with the only external dependency (the D3.js drawing library) pulled in from a single script tag. The backend is a small FastAPI application that starts with one command and serves both the static front end and the compiler's HTTP routes from the same process. The shape of the whole thing is shown below.

```
Browser (static/index.html, app.js, viz.js)
        |   source code sent as JSON, over HTTP
        v
FastAPI backend  (app/main.py)
        |   runs every phase below, in order, on one request
        v
lexer -> parser -> ast_nodes -> semantic -> tac -> optimize -> codegen
        |   produces one JSON object: tokens, tree, symbols,
        |   TAC, control-flow graph, optimisation log, assembly
        v
app/vm.py   (stack-based virtual machine)
        |   executes the assembly, on request
        v
Program output, shown back in the browser
```

The backend exposes four routes. GET / serves the front end itself. POST /compile is the important one: it takes the submitted source text, runs it through the lexer, the parser, the semantic analyser, the three-address code generator, the optimiser and the code generator in strict sequence, and sends back a single JSON object carrying the output of every one of those phases at the same time. POST /run takes a finished assembly listing and executes it on the virtual machine, returning whatever the program printed, so that running a program is a separate step from compiling it and does not force a full recompile every time. A couple of small auxiliary routes, GET /api/health and GET /api/demo, exist mainly so the front end can check that the backend is alive and load a ready-made example program.

The reason the whole response comes back in one object rather than the front end asking for each phase separately is simplicity. If the browser had to track which phase it currently has and request the next one itself, every new feature would also need new state-management code on the client side, and a mismatch between what the backend has computed and what the browser

thinks it has would become an entire class of bug on its own. Sending everything back at once means the front end's job is reduced to pure rendering: given the JSON object, draw the eight tabs. There is no orchestration logic in the browser to get wrong.

3. Module Description

The backend is organised as one small Python module per compiler phase, plus a couple of shared files. Each one is described below, in the order a program actually passes through them.

3.1 app/lexer.py — Lexical Analyser

The first phase of the pipeline is the lexical analyser. Its job is to turn the raw text of a MiniLang program into a flat list of tokens, where each token carries its type, its exact text, and the line and column at which it begins. The scanner is hand-written rather than built from a table of regular expressions, mainly because of what that buys later in the pipeline: when the scanner itself is the thing moving the cursor character by character, the line and column of every token fall out of the scan for free, instead of needing a second pass over the source text to recover them afterwards. The scanner follows the maximal-munch rule that every compiler-design course teaches — at every position it consumes the longest possible token, so <= is read as one operator rather than < followed by =, and an identifier such as integer is read in full before the keyword table is even consulted, so it is never mistaken for the keyword int followed by eger. Because the alphabet of MiniLang is restricted to ASCII, the character classification used to decide what counts as a letter or a digit is written out explicitly rather than relying on a Unicode-aware built-in check, which keeps the accepted character set exactly what the grammar says it should be.

3.2 app/ast_nodes.py — Abstract Syntax Tree Nodes

Every node the parser can build — a function declaration, an if statement, a binary operation, a literal — is defined here as a small data class. Two decisions in this file matter more than they look. First, every node stores its own line and column, because a compiler that cannot point to where something went wrong is not much use to the person debugging it. Second, every node knows how to turn itself into a JSON dictionary for the front end, and that conversion is written to match the grammar exactly rather than the shape that is most convenient to keep in memory — for instance, a function's name is stored as a plain string internally, but when it is drawn on the Syntax tab it is expanded back into its own identifier node, because a parse tree that collapses four distinct grammar symbols into one text label would be misleading to look at. Each child node is also tagged with the grammar role it plays

in its parent — left or right in a binary operation, condition, then-branch or else-branch in an if statement — so that an asymmetric construct like a - b cannot be confused with b - a once it has been drawn as a tree.

3.3 app/parser.py — Recursive-Descent Parser

This file is the recursive-descent parser that turns the token stream into the abstract syntax tree. There is no explicit operator-precedence table anywhere in it; precedence is expressed structurally, through the order in which the expression-parsing functions call each other, from the loosest-binding operator down to the tightest. A function that parses addition always calls the function that parses multiplication to obtain its operands, so a multiplication can never end up above an addition in the resulting tree, and the usual convention that multiplication binds tighter than addition falls out of this call chain rather than from any comparison of operator strength. Every binary level is written as a loop, which makes those operators left-associative, while the unary level calls itself recursively, which makes prefix operators such as ! and - right-associative and lets something like !!x parse correctly. In this version of the parser, a syntax error simply raises an exception and stops the parse; this is a deliberate baseline, kept this way on purpose so that the error-recovery work planned on top of it has something fixed to be measured against.

3.4 app/semantic.py — Semantic Analyser and Symbol Table

The semantic analyser walks the finished tree and does two things at once: it resolves every name against a symbol table, and it works out a type for every expression. The symbol table is the classical stack-of-scopes design — entering a block pushes a new table onto the stack, leaving it pops that table back off, and looking up a name walks the stack outward from the innermost scope to the outermost one. Declaring a new name, by contrast, only ever checks the innermost table, which is exactly what makes shadowing a variable in a nested block legal while redeclaring the same name twice in the same block is not. Functions and variables are kept in one shared namespace rather than two separate ones, which matches the presentation most compiler-design textbooks use and has a genuine, visible consequence: a variable named the same as a function will shadow it, and calling that name afterwards is correctly reported as calling something that is not callable. Types are computed bottom-up — each expression's type is a function of its children's types, worked out once the children have already been visited — which is the standard synthesized-attribute approach to type checking.

3.5 app/tac.py — Three-Address Code, Basic Blocks and the CFG

This module lowers the type-checked tree into three-address code, then groups that code into basic blocks and links the blocks into a control-flow graph. Three-address code means every instruction has at most two operands and one result, each of which is a variable, a constant, or a compiler-generated temporary such as t1 or t2 — this is the representation most textbooks use as the bridge between a tree and real machine code, because it is already flat and sequential without yet committing to any particular target machine. Almost every expression is translated the straightforward way, bottom-up, each subexpression leaving its value in a place that its parent then combines; the one exception is the && and || operators, which are translated using jumps instead, because short-circuit evaluation is a requirement of the language and not just a stylistic choice — the right-hand side of f() && g() must provably never execute when f() is already false. Basic blocks are found using the standard leader algorithm, with one extra rule added here: every label is treated as a leader unconditionally, which is what stops two functions placed one after another in the same instruction stream from bleeding into each other's blocks.

3.6 app/optimize.py — Optimisation Passes

The four optimisation passes live here, and the Optimisation tab lets each one be switched on or off independently so the effect of a single pass can be seen on its own. Constant folding replaces an instruction like t1 = 3 + 4 with t1 = 7 whenever both operands are already known at compile time. Constant propagation keeps track, within a single basic block, of which places are currently known to hold a constant value, and substitutes that value wherever the place is read next. Common-subexpression elimination keeps track of which computations have already been done earlier in the same block and rewrites a repeated computation into a copy of the earlier result. These three passes are deliberately local — they only ever look inside one basic block — because that is what keeps them simple and still correct, since there is only one path through a single block and nothing to branch around. Dead-code elimination is the odd one out: a value computed in one block is very often still needed in a different block reached through a branch or a loop, so this pass genuinely needs a backward liveness analysis computed over the whole control-flow graph, not a shortcut that only looks within one block.

3.7 app/codegen.py — Target Code Generation

This module takes the optimised three-address code and lowers it into the flat assembly that the virtual machine runs. The target is a simple stack machine: there is no accumulator and no general-purpose registers, only one operand stack that also doubles as the call stack. Each function's parameters and local variables are given a fixed position relative to a frame pointer, following the same activation-record layout a compilers textbook draws for a stack-based

runtime — parameters at negative offsets from the frame pointer, locals at non-negative ones, with the control link and the return address kept in between. Jump and call targets are written out symbolically first, using label names, and only resolved into concrete instruction addresses in a second pass once every label's final position is known; this is the classical two-pass assembler technique, and it means the virtual machine itself never has to look a name up by string at run time, only follow an address that has already been resolved.

3.8 app/vm.py — Stack-Based Virtual Machine

This file is the interpreter that actually executes the assembly app/codegen.py produces, and it is what the Run button on the Target tab calls. It keeps exactly two pieces of state beyond the stack itself: a program counter pointing at the instruction about to run, and a frame pointer marking the base of the current function's activation record. A function call pushes the caller's frame pointer and a return address onto the stack before jumping to the callee, and returning reads those two values back out before removing the frame — the same discipline that lets ordinary recursion, and even mutual recursion between two functions, work correctly without any special-case code anywhere. Division and modulo by a literal zero are deliberately left as a genuine runtime error here rather than something the optimiser is allowed to fold away at compile time, because that is exactly what those operations are supposed to do when they actually run.

3.9 app/literals.py — Shared Literal Handling

This is a small file, but an important one: it is the single place that decides how a MiniLang literal's text is turned into a value and back again. Three different phases — constant folding in app/optimize.py, three-address code generation in app/tac.py, and the virtual machine in app/vm.py — all need to agree on exactly how a number or a boolean is written and computed, including details like which way integer division truncates and which sign a modulo operation takes. Keeping that logic in one shared module, instead of three separate copies of it scattered across the codebase, is what guarantees a value folded at compile time and the same value computed at run time can never quietly disagree with each other.

3.10 app/main.py — FastAPI Application

This file is the FastAPI application that ties everything together and gives the browser something to talk to. Its most important route, POST /compile, runs every phase of the pipeline in order on the submitted source code and returns one JSON object holding the output of all of them at once — the token list, the parse tree, the symbol table, the three-address code, the

control-flow graph, the optimisation log, and the final assembly. A separate POST /run route takes that assembly and executes it on the virtual machine, returning whatever the program printed. This file deliberately does not try to be clever about only sending the front end the phase it currently needs; sending everything every time is simpler, and it is what makes the front end's job purely about rendering rather than about deciding what to ask for next.

3.11 Front End — index.html, app.js, viz.js, style.css

On the browser side, index.html lays out the eight tabs — Source, Lexical, Parser Theory, Syntax, Semantic, ICG, Optimisation and Target — and app.js is responsible for sending the source code to /compile and rendering whatever comes back into each tab. viz.js handles the two visualisations that are drawn rather than listed in a table: the parse tree and the control-flow graph, both built with D3.js. None of this needed a build step, a package manager, or a JavaScript framework — the files are loaded directly by the browser, and the only external library, D3, is loaded from a single script tag. That choice was made on purpose: a project that has to be demonstrated live on the day of evaluation should not depend on anything that can break between one machine and another.

4. Grammar of MiniLang

MiniLang's grammar is written below in the usual style a compiler-design course uses, with | separating alternatives and the usual meaning for *, + and ?. It is deliberately small — a language needs to cover function declarations, typed variables, the common control-flow statements and a full expression grammar to be worth calling a language, and MiniLang covers exactly that and nothing more.

```
program    -> funcDecl+
funcDecl   -> 'func' ID '(' params? ')' ':' type block
params     -> type ID (',' type ID)*
type       -> 'int' | 'float' | 'bool' | 'void'
block      -> '{' stmt* '}'

stmt       -> varDecl | assign | ifStmt | whileStmt
            | returnStmt | printStmt | exprStmt
varDecl    -> type ID ('=' expr)? ';'
assign     -> ID '=' expr ';'
ifStmt     -> 'if' '(' expr ')' block ('else' block)?
whileStmt  -> 'while' '(' expr ')' block
returnStmt -> 'return' expr? ';'
printStmt  -> 'print' '(' expr ')' ';'

expr       -> logicOr
logicOr    -> logicAnd ('||' logicAnd)*
```

```
logicAnd   -> equality ('&&' equality)*
equality   -> compare (('==' | '!=') compare)*
compare    -> term (('<' | '>' | '<=' | '>=') term)*
term       -> factor (('+' | '-') factor)*
factor     -> unary (('*' | '/' | '%') unary)*
unary      -> ('!' | '-') unary | primary
primary    -> NUM | ID | 'true' | 'false'
            | ID '(' args? ')' | '(' expr ')'
```

A few things about this grammar are worth calling out, because they are exactly the properties a predictive parser needs. First, there is no left recursion anywhere in it — a rule like term, which handles + and -, is written as factor followed by zero or more occurrences of (+ or -) factor, rather than as term followed by '+' and factor referring back to itself on the left. Left recursion is harmless for a human reading the grammar, but a recursive-descent function implementing a left-recursive rule would call itself before consuming any input and never terminate, so rewriting every left-recursive rule into this repetition form is a mechanical step most compiler-design textbooks cover early, and it is already built into MiniLang's grammar rather than left as something the parser has to work around afterwards.

Second, precedence and associativity are both expressed purely through the shape of the grammar rather than through any separate table. The chain expr -> logicOr -> logicAnd -> equality -> compare -> term -> factor -> unary -> primary lists the operators from loosest-binding to tightest-binding, and because each level only ever calls the level below it for its operands, an expression like 2 + 3 * 4 is forced to parse with the multiplication nested one level deeper than the addition, giving the tree (+ 2 (* 3 4)) and never the other grouping. The repetition inside each binary level is what makes these operators left-associative, since a chain like a - b - c is consumed left to right within the same loop; unary, by contrast, is written as a rule that can call itself again before reaching primary, which is what makes prefix operators right-associative and allows something like --x or !!x to parse at all.

Third, at every point where the parser has to choose between alternatives, the next single token is always enough to make that choice — the grammar never needs to look two tokens ahead or guess and backtrack. A statement, for example, is one of varDecl, assign, ifStmt, whileStmt, returnStmt, printStmt or exprStmt, and the very first token of the input — a type keyword, an identifier, if, while, return or print — tells the parser unambiguously which one it is looking at before it commits to a choice. This single-token-lookahead property is exactly the condition an LL(1) grammar has to satisfy, and it is also what makes the hand-written recursive-descent parser possible to write at all without any backtracking logic.

5. Algorithms

This section pulls together, in one place, the algorithms behind each phase, since the module descriptions above introduce them but a compiler-design report should state them plainly on their own.

5.1 Maximal-Munch Lexical Scanning

At each position in the source text, the scanner looks at the current character to decide broadly what kind of token is starting — a digit starts a number, a letter starts an identifier or a keyword, a punctuation character starts an operator or a delimiter — and then keeps consuming characters for as long as they still belong to that token. For operators that share a prefix, such as < and <=, the scanner always tries the longer match first, which is the maximal-munch rule every lexical-analysis chapter states as the tie-breaking principle a scanner must follow. Once a run of letters and digits has been consumed in full, it is looked up in a fixed table of keywords; if it is not found there, it is classified as an identifier. This whole process is, in essence, a hand-simulated deterministic finite automaton, the same kind of machine a regular-expression-based scanner would otherwise be compiled into.

5.2 Predictive (Recursive-Descent) Parsing

The parsing algorithm is predictive parsing by recursive descent, where every nonterminal in the grammar becomes one function, and every production's right-hand side becomes that function's body. A function for a nonterminal with several alternatives looks at the next token in the stream — without consuming it — and uses that one token to decide which alternative to follow, which is exactly what the grammar's LL(1) property, discussed in the grammar section, guarantees is always possible here. Parsing an expression works by precedence climbing: the function for the loosest-binding operator calls down to the function for the next tighter one to obtain each operand, loops to consume any repeated occurrences of its own operator, and returns. The algorithm never needs to backtrack, because at no point does more than one grammar alternative ever match the same next token.

5.3 Scope Resolution and Type Inference

Name resolution is done with a stack of hash tables representing nested scopes: entering a block pushes a fresh table, leaving it pops the table back off, and resolving a name searches outward from the top of the stack to the bottom until a matching entry is found. Type inference is done in the same tree walk, computed bottom-up as a synthesized attribute — each expression node's type is worked out purely from the already-computed types of its own children, following the

usual typing rules for arithmetic, comparison and logical operators, and the result is written back onto the node itself so later phases, and the Semantic tab, can read it off directly without recomputing anything.

5.4 Three-Address Code Generation

Translating the syntax tree into three-address code follows the synthesized-place scheme most compilers courses teach for expressions: each subexpression, once translated, leaves its value sitting in a place, and its parent expression's code simply refers to its children's places to build its own result in a new place. Statements are translated into the equivalent sequence of assignments, conditional jumps and labels — an if statement, for instance, becomes a conditional jump to a label marking the start of its else branch, or the end of the statement if there is no else, matching the control-flow translation scheme a compilers textbook sets out for conditionals. The one deliberate departure from pure value-based translation is for && and ||, which are translated using jumps so that the right-hand operand's code is only ever reached when it actually needs to be evaluated — this is what makes short-circuit evaluation a real, provable property of the generated code rather than something the source language merely promises.

5.5 Leader Algorithm and Control-Flow Graph Construction

Once a function's three-address code has been generated, it is split into basic blocks using the standard leader algorithm: an instruction is a leader if it is the very first instruction of the function, if it is the target of some jump anywhere in the code, or if it immediately follows a jump or a return instruction. Every instruction between one leader and the next, exclusive of the next leader itself, forms one basic block. Once the blocks are known, the control-flow graph is built by looking at how each block's last instruction behaves — an unconditional jump gives the block exactly one outgoing edge, a conditional jump gives it two, one to the jump's target and one to the block that follows it in the listing, and anything else simply falls through to the next block in sequence.

5.6 Local Optimisation Passes

Constant folding, constant propagation and common-subexpression elimination are all implemented as a single forward scan through one basic block at a time, each keeping a small table as it goes — folding simply evaluates an operation on the spot when both its operands are already literal constants; propagation remembers which place currently holds which constant and substitutes it on every later read within the same block; and common-subexpression

elimination remembers which computation has already produced which place and rewrites a repeated one into a plain copy. Because each of these three passes can create new opportunities for the other two — propagating a constant can make an operation suddenly foldable, and folding can make two originally different-looking computations become identical for elimination to catch — all three are run together, repeatedly, until a full pass over the code makes no further change, which is the fixed-point iteration pattern that shows up everywhere in optimisation theory.

5.7 Dead-Code Elimination as a Global Data-Flow Problem

Dead-code elimination cannot use the same single-block shortcut as the other three passes, because a value computed in one block is routinely still needed in a different block reached through a branch or the back edge of a loop. It is instead implemented as a backward liveness analysis over the entire control-flow graph: starting from the exit of each function and walking backward, the analysis repeatedly recomputes, for every block, the set of places whose current value might still be read somewhere further down some path, until the computed sets stop changing from one iteration to the next — again a fixed point, but over the whole graph this time rather than over one block. An instruction is only removed once it is proven that its result is not in that live set at the point right after it executes, and a function call is always kept regardless of whether anything reads its result, because a call can have a side effect, such as printing something, that has nothing to do with the value it returns.

5.8 Target Code Generation with Backpatching

Generating the final stack-machine assembly walks the three-address code once, instruction by instruction, emitting one or more assembly instructions for each — a binary operation becomes a pair of loads followed by an arithmetic opcode, for instance, and a function call becomes a sequence that pushes the arguments, pushes a control link and a return address, and then jumps. Every jump and call target is written out as a label name during this pass rather than as a resolved number, because the final address of a label — especially a label such as a loop's start, which is defined before some of the jumps that target it and after others — is not always known yet at the point where the jump to it is emitted. A second pass over the finished listing then resolves every label into its actual instruction index and rewrites every jump and call to use that number directly; this two-pass generate-then-backpatch technique is the standard way a classical assembler handles exactly this ordering problem.

5.9 Stack-Machine Execution Model

The virtual machine executes the backpatched assembly using one list as both its operand stack and its call stack, addressed through two registers: a program counter giving the index of the instruction about to run, and a frame pointer giving the base of the current function's activation record. Arithmetic instructions pop their operands off the top of the stack and push back a single result; a function call pushes a control link, the caller's own frame pointer, and a return address onto the stack before jumping to the callee's first instruction, and a return instruction reads both of those back out, restores them, and jumps back — which is precisely the discipline that makes recursion, including a function calling itself or two functions calling each other, work correctly without the virtual machine needing to know anything special about the shape of the call graph in advance.

6. Summary

Taken together, the modules, grammar and algorithms described in this report form one coherent pipeline rather than six independent exercises: a program's text is scanned once, parsed once into a tree whose shape already encodes operator precedence, checked once for scope and type errors, lowered once into three-address code and a control-flow graph, optimised by a mixture of local and global passes depending on what each pass actually needs to be correct, and finally assembled and executed on a small stack machine of our own design. Every one of those steps is an implementation of an algorithm a compiler-design course introduces on the board, carried through to a form that actually runs on real input and can be watched happening, phase by phase, in the browser.